

Technik Programista

Zaawansowane aplikacje webowe

Flask – modele i baza danych

SQLAlchemy, SQLite, CRUD

*Klasa = tabela, obiekt = wiersz. Dodawanie, odczyt, edycja, usuwanie.
Pełne MVC.*

Klasa 4 | rok szkolny 2026/2027 | System: Windows

Spis treści

Spis treści

Spis treści.....	2
1. Po co baza i po co ORM	3
2. Instalacja i konfiguracja	3
3. Pierwszy model	4
4. CRUD	4
4.1 Create – dodawanie.....	4
4.2 Read – odczyt.....	5
4.3 Update – edycja.....	5
4.4 Delete – usuwanie	5
6. Podgląd bazy	7
7. Porządek: models.py	7
8. Gdzie teraz jest MVC	8
9. Najczęstsze problemy	8
10. Zadania	9
Zadanie 1: Model z Twojej domeny	9
Zadanie 2: CRUD w konsoli.....	9
Zadanie 3: Lista i szczegóły z bazy	9
Zadanie 4: Dodawanie i usuwanie	9
Zadanie 5: Edycja	9
Zadanie 6: Filtrowanie	9
Zadanie 7: Logika w modelu	9
Zadanie 8: models.py	9
Zadanie 9: Dla chętnych – drugi model	9
Zadanie 10: Repo	10

1. Po co baza i po co ORM

Lista **PRODUKTY** w pamięci znika przy restarcie i nie działa, gdy serwer ma więcej niż jeden proces. **Baza danych** trwale przechowuje dane. Użyjemy **SQLite** – jednopliskowej bazy, bez instalowania serwera, wbudowanej w Pythona.

Do bazy można pisać czystym SQL-em. Można też użyć **ORM** (Object-Relational Mapping): biblioteki, która tłumaczy klasy Pythona na tabele, a obiekty na wiersze.

Czysty SQL	ORM (SQLAlchemy)
<code>SELECT * FROM produkt WHERE id = 5</code>	<code>Produkt.query.get(5)</code>
<code>INSERT INTO produkt (nazwa, cena) VALUES ('Mysz', 49)</code>	<code>db.session.add(Produkt(nazwa="Mysz", cena=49))</code>
<code>DELETE FROM produkt WHERE id = 5</code>	<code>db.session.delete(p)</code>
stringi, łatwo o SQL injection	obiekty, parametry uciekane automatycznie

ORM to jest **model** z MVC w najczystszej postaci: klasa opisuje, czym są dane. Django ma własny ORM o niemal identycznej składni – to, co tu zrobisz, przeniesiesz jeden do jednego.

 *SQL i tak musisz znać (bazy danych masz w podstawie programowej). ORM nie zastępuje SQL-a, tylko go ukrywa – warto od czasu do czasu zerknąć, jakie zapytanie wygenerował.*

2. Instalacja i konfiguracja

```
pip install flask-sqlalchemy
pip freeze > requirements.txt
```

W `app.py`, zaraz po utworzeniu `app`:

```
from flask import Flask, render_template, request, redirect, url_for
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///sklep.db"
db = SQLAlchemy(app)
```

- `sqlite:///sklep.db` – plik `sklep.db` w folderze `instance/` obok `app.py`. Flask tworzy go sam.
- `db` – obiekt, przez który robimy wszystko: definiujemy modele, zapisujemy, pytamy.

 **Dodaj do .gitignore: instance/ oraz *.db. Baza z danymi testowymi nie idzie do repozytorium.**

3. Pierwszy model

```
class Produkt(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    nazwa = db.Column(db.String(100), nullable=False)
    cena = db.Column(db.Float, nullable=False)
    dostepny = db.Column(db.Boolean, default=True)

    def __repr__(self):
        return f"<Produkt {self.nazwa}>"

with app.app_context():
    db.create_all()
```

Element	Znaczenie
<code>class Produkt(db.Model)</code>	klasa = tabela produkt (nazwa tabeli: małe litery)
<code>db.Column(...)</code>	atrybut klasy = kolumna tabeli
<code>db.Integer, db.String(100), db.Float, db.Boolean, db.DateTime, db.Text</code>	typy kolumn
<code>primary_key=True</code>	klucz główny, numerowany automatycznie
<code>nullable=False</code>	pole wymagane (NOT NULL)
<code>default=True</code>	wartość domyślna
<code>__repr__</code>	jak obiekt ma się wyświetlać w konsoli – do debugowania
<code>db.create_all()</code>	utwórz tabele dla wszystkich modeli (jeśli nie istnieją)

⚠ **create_all tworzy tabele, ale NIE modyfikuje istniejących. Dodałeś kolumnę do klasy? Usuń plik .db i uruchom ponownie (na etapie nauki) – albo użyj migracji (Flask-Migrate, poznasz w Django jako makemigrations).**

4. CRUD

Cztery operacje, które ma każda aplikacja z danymi. Wypróbuj w konsoli: **flask shell** (venv aktywny, w folderze projektu) – masz tam dostęp do **db** i modeli.

4.1 Create – dodawanie

```
p = Produkt(nazwa="Laptop", cena=2999.99)
db.session.add(p)
db.session.commit()
print(p.id)          # 1 – nadane przez bazę po commicie
```

session to poczekalnia (jak staging w Gicie). **add** wkłada, **commit** zapisuje na dysk. Bez **commit** nic się nie zapisze.

4.2 Read – odczyt

```

Produkt.query.all()           # lista wszystkich
Produkt.query.get(1)          # po id, None jeśli brak
Produkt.query.get_or_404(1)   # po id, 404 jeśli brak
Produkt.query.filter_by(dostepny=True).all() # z warunkiem
Produkt.query.filter(Produkt.cena < 100).all()
Produkt.query.order_by(Produkt.cena).all()
Produkt.query.filter_by(nazwa="Mysz").first()
Produkt.query.count()

```

4.3 Update – edycja

```

p = Produkt.query.get(1)
p.cena = 2799
db.session.commit()

```

Zmieniasz atrybut obiektu, robisz **commit**. Nie ma osobnego „update”.

4.4 Delete – usuwanie

```

p = Produkt.query.get(1)
db.session.delete(p)
db.session.commit()

```

5. Pełna aplikacja

Łączymy wszystko z materiałów 04–07. Kontrolery:

```

@app.route("/")
def lista():
    produkty = Produkt.query.order_by(Produkt.nazwa).all()
    return render_template("lista.html", produkty=produkty)

@app.route("/produkt/<int:id>")
def szczegoly(id):
    p = Produkt.query.get_or_404(id)
    return render_template("produkt.html", p=p)

@app.route("/dodaj", methods=["GET", "POST"])
def dodaj():
    if request.method == "POST":
        p = Produkt(nazwa=request.form["nazwa"],
                    cena=float(request.form["cena"]))
        db.session.add(p)
        db.session.commit()
        return redirect(url_for("lista"))
    return render_template("dodaj.html")

@app.route("/usun/<int:id>", methods=["POST"])
def usun(id):
    p = Produkt.query.get_or_404(id)
    db.session.delete(p)
    db.session.commit()
    return redirect(url_for("lista"))

```

templates/lista.html:

```
{% extends "base.html" %}
{% block tresc %}
<h1>Produkty ({{ produkty|length }})</h1>
{% if produkty %}
<table>
{% for p in produkty %}
  <tr>
    <td>{{ loop.index }}</td>
    <td>
      <a href="{{ url_for('szczegoly', id=p.id) }}">{{ p.nazwa }}</a>
    </td>
    <td>{{ p.cena }} zł</td>
    <td>{% if not p.dostepny %}<b>niedostepny</b>{% endif %}</td>
    <td>
      <form method="post" action="{{ url_for('usun', id=p.id) }}">
        <button>Usuń</button>
      </form>
    </td>
  </tr>
{% endfor %}
</table>
{% else %}
<p>Brak produktów. <a href="{{ url_for('dodaj') }}">Dodaj pierwszy</a>.</p>
{% endif %}
{% endblock %}
```

templates/produkt.html:

```
{% extends "base.html" %}
{% block tresc %}
<h1>{{ p.nazwa }}</h1>
<p>Cena: {{ p.cena }} zł</p>
<p>Dostepny: {{ "tak" if p.dostepny else "nie" }}</p>
<a href="{{ url_for('lista') }}">Wróć</a>
{% endblock %}
```

templates/dodaj.html – formularz z materiału 06, bez zmian.

- Szablony **nie zmieniły się** względem wersji ze słownikami – **p.nazwa** działa i dla słownika, i dla obiektu. To jest zysk z rozdzielenia warstw.
- Usuwanie przez POST, nie przez link GET. Link GET może wywołać robot indeksujący albo podgląd w komunikatorze – i usunie produkt.

6. Podgląd bazy

Plik `instance/sklep.db` można otworzyć programem **DB Browser for SQLite** (<https://sqlitebrowser.org>, darmowy). Zobaczysz tabelę `produkt`, kolumny i wiersze – dokładnie to, co zdefiniowałeś w klasie. Zakładka `*Execute SQL*` pozwala wpisać `SELECT * FROM produkt` i porównać z tym, co robi ORM.

Żeby zobaczyć, jaki SQL generuje SQLAlchemy:

```
app.config["SQLALCHEMY_ECHO"] = True
```

Każde zapytanie wypisze się w terminalu. Wyłącz, jak już zobaczysz – zaśmieca log.

7. Porządek: models.py

Wszystko w `app.py` działa, ale przy trzech modelach plik ma 300 linii. Rozdzielmy:

```
flask-sklep/
├── app.py          <- konfiguracja + kontrolery
├── models.py       <- modele (M)
├── templates/      <- widoki (V)
├── static/
├── instance/sklep.db
├── requirements.txt
└── README.md
```

`models.py`:

```
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()

class Produkt(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    nazwa = db.Column(db.String(100), nullable=False)
    cena = db.Column(db.Float, nullable=False)
    dostepny = db.Column(db.Boolean, default=True)

    def cena_brutto(self):
        return round(self.cena * 1.23, 2)
```

`app.py`:

```
from flask import Flask, render_template, request, redirect, url_for
from models import db, Produkt

app = Flask(__name__)
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///sklep.db"
db.init_app(app)

with app.app_context():
    db.create_all()

# ... kontrolery bez zmian ...
```

Zwróć uwagę na `cena_brutto()` – **logika biznesowa w modelu**, nie w kontrolerze ani w szablonie. W szablonie piszesz `{{ p.cena_brutto() }}`. Gdy zmieni się VAT, poprawiasz jedno miejsce.

💡 Dokładnie tak wygląda Django: `models.py`, `views.py` (= nasze kontrolery), `templates/`. Przechodząc na Django, zmienisz składnię, nie sposób myślenia.

8. Gdzie teraz jest MVC

Warstwa	U nas	Plik
Model	klasy <code>db.Model</code> + metody z logiką (<code>cena_brutto</code>)	<code>models.py</code>
View	szablony Jinja2	<code>templates/*.html</code>
Controller	funkcje z <code>@app.route</code> : pobierz z modelu, oddaj szablonowi, przekieruj	<code>app.py</code>

Mamy komplet. Kontroler `lista()` ma 2 linie. Szablon nie wie, skąd są dane. Model nie wie, że istnieje HTML. Test: czy dałoby się do tego modelu dopiąć aplikację na Androida zamiast szablonów? Tak – i to zrobimy po Django i Reakcie.

9. Najczęstsze problemy

Objaw	Rozwiązanie
<code>RuntimeError: Working outside of application context</code>	<code>db.create_all()</code> (i każda praca z bazą poza trasą) musi być w <code>with app.app_context():</code>
<code>no such table: produkt</code>	tabela nie została utworzona – sprawdź, czy <code>create_all()</code> jest wywołane po zdefiniowaniu klas
<code>no such column: produkt.opis</code>	dodałeś kolumnę po utworzeniu bazy. Usuń <code>instance/sklep.db</code> , uruchom ponownie
<code>IntegrityError: NOT NULL constraint failed</code>	próbujesz zapisać obiekt bez pola <code>nullable=False</code>
dane się nie zapisują	brak <code>db.session.commit()</code>
<code>ValueError: could not convert string to float</code>	pole <code>cena</code> z formularza jest puste lub ma przecinek zamiast kropki
<code>.db</code> wylądował na GitHubie	<code>.gitignore</code> z <code>instance/</code> i <code>*.db</code> ; <code>git rm --cached instance/sklep.db</code>

10. Zadania

Zadanie 1: Model z Twojej domeny

Zastąp listę słowników z materiału 06 klasą `db.Model`. Minimum 4 pola różnych typów, w tym jedno `Boolean` i jedno `nullable=False`. Uruchom, sprawdź w DB Browser, że tabela istnieje.

Zadanie 2: CRUD w konsoli

W `flask shell` dodaj 3 obiekty, wypisz wszystkie, zmień jedno pole, usuń jeden obiekt. Wklej przebieg sesji do README (sekcja „Test CRUD”).

Zadanie 3: Lista i szczegóły z bazy

Trasy `/` i `/element/<int:id>` czytają z bazy. `get_or_404` dla nieistniejącego id. Lista posortowana po nazwie.

Zadanie 4: Dodawanie i usuwanie

Formularz POST dodający do bazy; przycisk usuwania (POST) przy każdym elemencie. Po obu – redirect.

Zadanie 5: Edycja

Trasa `/edytuj/<int:id>` GET + POST. Formularz wypełniony aktualnymi wartościami (`value="{{ p.nazwa }}"`). Po zapisie redirect na szczegóły.

Zadanie 6: Filtrowanie

Na liście: `?tylko_dostepne=1` pokazuje tylko elementy z polem logicznym `True` (`filter_by`). Wyszukiwarka `?q=` po nazwie (`filter(Model.nazwa.contains(q))`).

Zadanie 7: Logika w modelu

Dodaj do modelu metodę z logiką z Twojej domeny (cena brutto, czy termin minął, czy rezerwacja jest aktywna, liczba dni od publikacji). Użyj jej w szablonie. Nie licz tego w kontrolerze ani w szablonie.

Zadanie 8: models.py

Rozdziel projekt według rozdziału 7. README: struktura projektu i tabela „warstwa MVC → plik”.

Zadanie 9: Dla chętnych – drugi model

`Kategoria` (id, nazwa) i relacja: element należy do kategorii (`db.ForeignKey`, `db.relationship`). Lista z nazwą kategorii przy każdym elemencie.

Zadanie 10: Repo

Commit `Flask: SQLAlchemy, model, CRUD`. Sprawdź, że `instance/` i `*.db` nie ma na GitHubie.